UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

título del TFG
Documentación Técnica

Presentado por nombre alumno
en Universidad de Burgos — 12 de mayo de 2026
Tutor: nombre tutor

Índice general

Índice general **i**

Índice de figuras **iii**

Índice de tablas **iv**

Apéndice A Plan de Proyecto Software **1**
A.1. Introducción 1
A.2. Planificación temporal 2
A.3. Estudio de viabilidad 8

Apéndice B Especificación de Requisitos **11**
B.1. Introducción 11
B.2. Objetivos generales 11
B.3. Catálogo de requisitos 12
B.4. Especificación de requisitos 16

Apéndice C Especificación de diseño **29**
C.1. Introducción 29
C.2. Diseño de datos 29
C.3. Diseño arquitectónico 29
C.4. Diseño procedimental 29

Apéndice D Documentación técnica de programación **31**
D.1. Introducción 31
D.2. Estructura de directorios 31
D.3. Manual del programador 31

D.4. Compilación, instalación y ejecución del proyecto 31
D.5. Pruebas del sistema 31

Apéndice E Documentación de usuario 33
E.1. Introducción 33
E.2. Requisitos de usuarios 33
E.3. Instalación 33
E.4. Manual del usuario 33

Apéndice F Anexo de sostenibilización curricular 35
F.1. Introducción 35

Bibliografía 41

Índice de figuras

Índice de tablas

A.1. Resumen de Sprints del proyecto 2
A.2. Coste de personal 8
A.3. Coste de recursos hardware (amortizado a 4 meses) 9
A.4. Coste total del proyecto 9
A.5. Licencias de las tecnologías utilizadas 10

B.1. A01 - Paciente. 16
B.2. A02 - Administrador. 17
B.3. A03 - Motor de clasificación. 18
B.4. A04 - Módulo de reconocimiento de voz. 18
B.5. A05 - Almacenamiento local. 19
B.6. CU-01 Introducir síntomas por voz. 20
B.7. CU-02 Introducir síntomas por texto. 21
B.8. CU-03 Obtener especialidad recomendada. 22
B.9. CU-04 Aportar síntomas adicionales y derivación. 23
B.10. CU-05 Añadir síntomas y especialidad. 24
B.11. CU-06 Visualizar log de incidencias. 25
B.12. CU-07 Resolver casos del log. 26
B.13. CU-08 Reentrenar modelos. 27

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La gestión de este proyecto software ha seguido una metodología ágil, organizando el trabajo en *Sprints* que permiten un desarrollo incremental y adaptativo. Dado que el proyecto se desarrolla de forma individual como Trabajo de Fin de Grado, la metodología adoptada es una variante simplificada de *Scrum*: se mantienen los Sprints como unidad de planificación y entrega, pero se prescinde de roles adicionales y de ceremonias formales como *daily standups*.

El seguimiento de tareas se ha realizado mediante un tablero Kanban gestionado a través de la plataforma *Zube.io* [6], integrada con el repositorio de GitHub. Las tarjetas se organizan en los estados propios de Zube.io: *Inbox* (entrada por defecto), *Backlog*, *Ready*, *In Progress*, *In Review* y *Done*, permitiendo un control visual del avance de cada Sprint. No se han utilizado Epics; en su lugar, los Sprints agrupan directamente las tareas relacionadas con cada componente del sistema.

El proyecto se articula en torno a cuatro bloques técnicos de modelado —*Term Frequency – Inverse Document Frequency* (TF-IDF) como línea base, TF-IDF con reducción *Singular Value Decomposition* (SVD), un clasificador *Support Vector Machine* (SVM) y una Red Neuronal Artificial (RNA) de tipo *Multi-layer Perceptron* (MLP)— a los que se añaden las fases de preparación de datos, desarrollo del frontal de usuario, panel de administración y documentación. La memoria y los anexos se han elaborado de forma paralela al desarrollo.

Sprint	Inicio	Fin
Introducción y Objetivos	19/02/2026	03/03/2026
Manejo y Edición con TeXTStudio	19/02/2026	03/03/2026
Búsqueda y conversión de DataSheets con datos	26/02/2026	23/04/2026
Creación de Script TF-IDF	01/04/2026	23/04/2026
Creación de Front End	21/04/2026	23/04/2026
Creación de Script TFIDF-SVD	22/04/2026	06/05/2026
Creación de Script SVM	26/04/2026	06/05/2026
Teoría del TFG — Memoria	03/03/2026	06/06/2026
Trabajos sobre el Front END	22/04/2026	01/06/2026
Adaptaciones de Código y Nuevas Funcionalidades	05/05/2026	01/06/2026
Anexos del TFG	06/05/2026	01/06/2026

Tabla A.1: Resumen de Sprints del proyecto

A.2. Planificación temporal

Visión general de los Sprints

A continuación se recoge la relación completa de Sprints realizados, con sus fechas de inicio y fin:

Detalle de los Sprints

Sprint 1 — Introducción y Objetivos

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint inaugural dedicado a establecer la base documental del proyecto. Se definió la estructura de la memoria en LaTeX y se redactó el capítulo de introducción —incluyendo el análisis de la crisis de accesibilidad sanitaria, el estado del arte y la justificación tecnológica— así como la formulación de los objetivos generales, funcionales y no funcionales. El cierre se produjo tres días después de la fecha planificada, motivado por la curva de aprendizaje inicial con el entorno de documentación.

Sprint 2 — Manejo y Edición con TeXTStudio

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint paralelo al primero, dedicado a la adquisición de competencia en TeXTStudio como herramienta principal de documentación técnica. Se trabajó en el manejo de abreviaturas y glosarios, la gestión de referencias bibliográficas con BibTEX, la inserción de figuras y tablas, y la compilación del documento completo. El cierre se adelantó dos días respecto a la fecha planificada.

Sprint 3 — Búsqueda y Conversión de DataSheets con Datos

Periodo: 26 de febrero de 2026 – 23 de abril de 2026.

Sprint dedicado a la localización, evaluación y transformación del conjunto de datos clínicos, en línea con el objetivo funcional de **preparación de los datos**. Los datos de partida consisten en un CSV con tres columnas: descripción de síntomas, enfermedad asociada y especialidad médica. Las tareas realizadas fueron:

- *Búsqueda de Datasheets:* exploración y evaluación de fuentes de datos clínicos públicas con licencia adecuada para el entrenamiento del clasificador.
- *Formato y Traducción:* normalización del formato de los datos, traducción y homogeneización de etiquetas de categorías médicas.
- *Aclaración de fuentes:* documentación de las fuentes seleccionadas, criterios de inclusión/exclusión y características del conjunto de datos definitivo.

Este sprint supuso la mayor desviación temporal del proyecto, cerrándose con casi siete semanas de retraso sobre lo planificado, debido a la escasez de *datasets* clínicos en español con las características requeridas.

Sprint 4 — Creación de Script TF-IDF

Periodo: 1 de abril de 2026 – 23 de abril de 2026.

Sprint centrado en el desarrollo del primer clasificador, que actúa como línea base dentro del objetivo funcional de **clasificación con cuatro modelos**. Se utilizó scikit-learn [3] como librería principal. Las tareas realizadas fueron:

- *Preparación del Entorno:* configuración del entorno de desarrollo Python con las dependencias necesarias (scikit-learn, pandas, numpy).

- *Pruebas en Google Colab:* validación inicial del *pipeline* de Procesamiento de Lenguaje Natural (PLN) y del vectorizador TF-IDF en un entorno de experimentación rápida.
- *Programación:* implementación completa del *pipeline* de preprocesado (normalización, eliminación de *stopwords*, tokenización y generación de bigramas y trigramas) y del clasificador TF-IDF base.
- *A Python y Streamlit tf_idf:* integración del script de clasificación TF-IDF con el entorno de ejecución local y primeras pruebas con Streamlit [5].
- *Separar el módulo que genera el modelo:* refactorización del código para desacoplar la generación y persistencia del modelo de la lógica de predicción, en línea con el objetivo no funcional de **modularidad**.

El sprint se cerró un día antes de lo planificado.

Sprint 5 — Creación de Front End

Periodo: 21 de abril de 2026 – 23 de abril de 2026.

Sprint dedicado al desarrollo de la interfaz de usuario con Streamlit [5], en línea con el objetivo funcional de **predicción a partir de texto libre**:

- *Creación de app.py:* desarrollo del frontal que permite al paciente introducir síntomas por escrito y recibir la especialidad médica predicha junto con el porcentaje de confianza. Se incorporó también la lógica de umbral: cuando la confianza queda por debajo del valor configurado, el sistema solicita al paciente que amplíe la descripción o elige entre las alternativas más probables.

El sprint se cerró con cinco días de antelación respecto a la fecha planificada.

Sprint 6 — Creación de Script TFIDF-SVD

Periodo: 22 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al segundo clasificador del objetivo funcional de **clasificación con cuatro modelos**: TF-IDF combinado con reducción de dimensionalidad mediante TruncatedSVD, implementado con scikit-learn [3]. Las tareas realizadas fueron:

- *Separar generar Módulo:* desacoplamiento del módulo de generación del modelo SVD para garantizar la persistencia de transformaciones y la consistencia entre entrenamiento y predicción.

- *Implementación en el app.py:* integración del modelo TF-IDF+SVD en el frontal Streamlit, permitiendo seleccionar el modelo de clasificación activo.

- *Creación del Script:* implementación de TruncatedSVD sobre la matriz TF-IDF, selección del número de componentes mediante validación cruzada y evaluación del rendimiento (F1 macro/micro, tiempo de entrenamiento y consumo de memoria) frente al modelo base.

El sprint se cerró con una semana de retraso sobre lo planificado, debido al tiempo requerido para el análisis sistemático de la varianza explicada en función del número de componentes.

Sprint 7 — Creación de Script SVM

Periodo: 26 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al tercer clasificador del objetivo funcional de **clasificación con cuatro modelos**: un SVM lineal entrenado con scikit-learn [3]. Las tareas realizadas fueron:

- *Creación:* implementación del clasificador SVM sobre las representaciones TF-IDF y TF-IDF+SVD, con búsqueda de hiperparámetros (`C`, `class_weight`) mediante `GridSearchCV` y validación cruzada estratificada optimizando macro-F1.

- *Implementación en el Front End:* integración del modelo SVM en el frontal Streamlit como opción de clasificación seleccionable.

- *Separar Generar Modelo:* refactorización para desacoplar la generación y persistencia del modelo SVM del módulo de predicción.

El sprint se cerró cuatro días antes de la fecha planificada.

Sprint 8 — Teoría del TFG — Memoria

Periodo: 3 de marzo de 2026 – 6 de junio de 2026.

Sprint de larga duración, planificado en paralelo con el desarrollo, que cubre la redacción del cuerpo teórico de la memoria. Las tareas contempladas son:

- *Búsqueda de Teoría en la red:* recopilación y análisis de fuentes bibliográficas sobre PLN, clasificación de texto clínico, TF-IDF, SVD, SVM y MLP.
- *Conceptos Teóricos:* redacción del capítulo de fundamentos teóricos de la memoria.
- *Añadir Referencias Teoría:* incorporación y revisión de las referencias bibliográficas al documento.
- *Convertir a LaTeX:* conversión y maquetación del contenido teórico al formato LaTeX de la plantilla del TFG.
- *Técnicas y Herramientas:* redacción del capítulo de técnicas y herramientas utilizadas.
- *Aspectos relevantes del desarrollo del proyecto:* redacción del capítulo de aspectos relevantes del desarrollo.
- *Trabajos Relacionados:* análisis y redacción del capítulo de trabajos relacionados y estado del arte.
- *Conclusiones y Líneas de trabajo Futuras:* redacción del capítulo de conclusiones y trabajo futuro.

Sprint 9 — Trabajos sobre el Front END

Periodo: 22 de abril de 2026 – 1 de junio de 2026.

Sprint que extiende el frontal Streamlit [5] con la integración progresiva de los modelos y la mejora de la experiencia de usuario. Las tareas contempladas son:

- *Inclusión TFIDF:* integración del modelo TF-IDF en la interfaz como opción de clasificación seleccionable.
- *Inclusión SVD:* integración del modelo TF-IDF+SVD en la interfaz.
- *Inclusión SVM en Front End:* integración del clasificador SVM en la interfaz.

- *Añadir Síntomas - Especialidad en Front End:* mejora de la interfaz para mostrar la especialidad médica asociada a cada predicción, incluyendo el indicador de confianza y la lógica de segunda vuelta cuando la predicción queda por debajo del umbral configurado.

Sprint 10 — Adaptaciones de Código y Nuevas Funcionalidades

Periodo: 5 de mayo de 2026 – 1 de junio de 2026.

Sprint que incorpora el cuarto clasificador y el panel de administración, completando así los objetivos funcionales de **clasificación con cuatro modelos** y **panel de administración**. Las tareas contempladas son:

- *Creación del Módulo Red Neuronal:* implementación del cuarto clasificador, una RNA de tipo MLP entrenada con *early stopping* y regularización para evitar sobreajuste. Este modelo completa el conjunto de cuatro enfoques comparados bajo el mismo protocolo de evaluación.
- *Creación Módulo Administrador:* desarrollo del panel de administración con tres herramientas: añadir nuevos pares síntoma-especialidad a un CSV ampliado, revisar y etiquetar el log de consultas no clasificadas con confianza suficiente, y disparar el reentrenamiento de los cuatro modelos con los datos actualizados.
- *Securizar el Módulo Administrador:* implementación de mecanismos de autenticación y control de acceso al panel de administración para restringir su uso al perfil autorizado.

Sprint 11 — Anexos del TFG

Periodo: 6 de mayo de 2026 – 1 de junio de 2026.

Sprint dedicado íntegramente a la redacción de los anexos de la memoria. Las tareas contempladas son:

- *Plan de Proyecto:* redacción del presente apéndice A.
- *Especificación de Requisitos:* redacción del apéndice B con el catálogo de requisitos funcionales y no funcionales.
- *Diseño:* redacción del apéndice C con la especificación de diseño arquitectónico y de datos.

- *Manual del Programador:* redacción del apéndice D con la documentación técnica de programación.
- *Manual del Usuario:* redacción del apéndice E con la guía de instalación y uso de la aplicación.
- *Anexo de sostenibilización curricular:* redacción del apéndice F con la reflexión sobre sostenibilidad.

A.3. Estudio de viabilidad

Viabilidad económica

El proyecto se ha desarrollado íntegramente sobre tecnologías de software libre y código abierto, por lo que el coste de licencias es nulo. A continuación se simula el coste real que supondría el desarrollo en un contexto empresarial.

Costes de personal

Se simula la contratación de un ingeniero informático junior durante los aproximadamente 4 meses de desarrollo efectivo (febrero–mayo de 2026). El salario de referencia se ha obtenido del XVIII Convenio Colectivo Estatal de Empresas de Consultoría y Tecnologías de la Información [2], y los tipos de cotización corresponden a los publicados por la Seguridad Social [4] y la Agencia Tributaria [1] para el ejercicio 2026.

Concepto	**Importe**
Salario mensual bruto	1.800,00 €
Retención IRPF (13,55 %)	243,90 €
Cotización SS empresa (36,25 %)	652,50 €
Coste mensual para la empresa	**2.452,50 €**
Total 4 meses	**9.810,00 €**

Tabla A.2: Coste de personal

Costes de recursos hardware

Se considera únicamente el equipo de trabajo del desarrollador (trabajo desde domicilio), calculando el coste amortizado para los 4 meses de duración del proyecto.

Concepto	Precio sin IVA	Coef.	Coste 4 meses
Ordenador portátil	750,00 €	26 %	65,00 €
Monitor adicional	160,00 €	30 %	16,00 €
Teclado y ratón	30,00 €	30 %	3,00 €
Total	**940,00 €**	—	**84,00 €**

Tabla A.3: Coste de recursos hardware (amortizado a 4 meses)

Coste total del proyecto

Concepto	Coste
Coste de personal (4 meses)	9.810,00 €
Coste de hardware (amortizado)	84,00 €
Herramientas software	0,00 €
TOTAL	**9.894,00 €**

Tabla A.4: Coste total del proyecto

Beneficios y modelo de explotación

Aunque el prototipo se desarrolla como TFG sin ánimo de lucro inmediato, se identifican dos vías de explotación potencial en un contexto empresarial:

- **Servicio SaaS para centros de salud:** la aplicación se ofrecería como servicio alojado con tarifas por volumen de consultas clasificadas (p. ej., cuota mensual por centro).
- **Solución standalone con soporte:** distribución del sistema empaquetado (p. ej., contenedor Docker) con diferentes niveles de soporte técnico y actualización de modelos con los datos propios de cada centro.

Viabilidad legal

El proyecto se apoya exclusivamente en librerías de código abierto con licencias compatibles entre sí y con el uso académico y comercial. Además, al ejecutarse en local sin enviar datos a servicios externos, el sistema es

compatible por diseño con los principios de minimización y soberanía del dato del **Reglamento General de Protección de Datos (RGPD)**.

Tecnología	**Licencia**
Python 3.11	PSF License (permisiva, uso comercial permitido)
scikit-learn, numpy, pandas [3]	BSD 3-Clause (permisiva, uso comercial permitido)
Streamlit [5]	Apache License 2.0 (permisiva, uso comercial permitido)
LaTeX / TeXTStudio	LPPL / GPL (libre para uso académico y personal)
Zube.io [6]	Propietaria, plan gratuito para proyectos individuales
Git	GPL v2 (libre)

Tabla A.5: Licencias de las tecnologías utilizadas

La licencia más restrictiva del conjunto es la GPL, presente solo en herramientas de desarrollo que no forman parte del artefacto distribuible. El código fuente de la aplicación puede distribuirse bajo licencia **MIT**, al no incorporar componentes GPL en el producto final.

En cuanto a los datos, el conjunto utilizado para el entrenamiento procede de fuentes públicas con licencias que permiten su uso en investigación y proyectos académicos. Para un despliegue en producción con datos reales de pacientes, sería necesario cumplir adicionalmente con la **Ley Orgánica de Protección de Datos (LOPD-GDD)**, garantizando el consentimiento informado y la anonimización de los registros clínicos.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este anexo se recoge la especificación de requisitos del prototipo *stand alone* desarrollado para la clasificación de síntomas en lenguaje natural y la asignación de la especialidad médica correspondiente. El sistema, ejecutado de forma local y sin dependencias de servicios externos, persigue dar respuesta a la problemática del triaje y de la asignación de citas descrita en el capítulo introductorio de la memoria.

Este documento tiene un objetivo doble: por un lado, servir como referencia funcional para la fase de análisis y diseño del software; por otro, recoger de forma estructurada las funcionalidades y restricciones del sistema, de manera que sea posible verificar su cumplimiento en fases posteriores de validación y pruebas.

La aplicación se compone de dos pestañas claramente diferenciadas: una orientada al **paciente**, que permite la introducción de síntomas mediante voz o texto y devuelve la especialidad predicha; y otra orientada al **administrador**, que permite la gestión del catálogo de síntomas, la revisión de los casos no resueltos y el reentrenamiento de los modelos de clasificación (TF-IDF, SVD, SVM y RNA).

B.2. Objetivos generales

El proyecto persigue alcanzar los siguientes objetivos generales, que orientan la definición de los requisitos:

- **Clasificación automática de síntomas en lenguaje natural:** desarrollar un sistema capaz de procesar descripciones libres del paciente (introducidas por teclado o por voz) y devolver una especialidad médica asociada, junto con un indicador de confianza.

- **Comparativa de modelos de aprendizaje:** integrar y comparar bajo un mismo protocolo experimental cuatro enfoques: TF-IDF (línea base), TF-IDF + SVD (reducción de dimensionalidad), SVM (clasificador supervisado) y una red neuronal artificial (RNA), permitiendo evaluar el equilibrio entre precisión y coste computacional.

- **Mecanismo de fallback ante baja confianza:** implementar un flujo de actuación cuando la predicción no alcanza un umbral mínimo de precisión, solicitando síntomas adicionales al paciente y, si tampoco se logra una clasificación fiable, derivando el caso a Atención General y registrándolo en un *log* para revisión posterior.

- **Gestión administrativa del conocimiento:** ofrecer al administrador la capacidad de ampliar el conjunto de datos de entrenamiento mediante la incorporación de nuevos pares síntoma–especialidad, así como de revisar y resolver los casos almacenados en el *log* de incidencias.

- **Reentrenamiento bajo demanda:** permitir al administrador regenerar los modelos de clasificación a partir del conjunto de datos actualizado, garantizando que las nuevas aportaciones se reflejen en futuras predicciones.

- **Ejecución local y privacidad por diseño:** garantizar que toda la operativa (datos, modelos e inferencia) se ejecute en el equipo del usuario, sin transmisión a servicios cloud, en línea con el paradigma de IA frugal.

- **Modularidad y extensibilidad:** diseñar la aplicación con una arquitectura modular que facilite el mantenimiento y la incorporación futura de nuevas funcionalidades, como canales de entrada alternativos (telefonía) o nuevos modelos.

B.3. Catálogo de requisitos

A continuación se enumeran los requisitos del sistema, separados en dos categorías: requisitos funcionales y requisitos no funcionales.

Requisitos funcionales

- **RF-1 Gestión del paciente:** la aplicación debe ofrecer una pestaña destinada al paciente que permita la introducción de síntomas y la consulta de la especialidad asignada.
  - **RF-1.1 Entrada de síntomas por texto:** el paciente debe poder introducir sus síntomas escribiéndolos directamente en un campo de texto.
  - **RF-1.2 Entrada de síntomas por voz:** el paciente debe poder dictar sus síntomas mediante un módulo de reconocimiento de voz, que transcribe el audio a texto antes de procesarlo.
  - **RF-1.3 Predicción de especialidad:** el sistema debe devolver al paciente la especialidad médica recomendada en función de los síntomas descritos.
  - **RF-1.4 Mostrar porcentaje de confianza:** la aplicación debe poder mostrar (de forma configurable) el porcentaje de confianza asociado a la predicción.
  - **RF-1.5 Solicitud de síntomas adicionales:** si la confianza de la predicción es inferior al umbral configurado, el sistema debe solicitar al paciente síntomas adicionales antes de emitir un diagnóstico definitivo.
  - **RF-1.6 Derivación a Atención General:** si tras la solicitud de síntomas adicionales la confianza sigue sin alcanzar el umbral, el sistema debe derivar el caso a la categoría de Atención General.
  - **RF-1.7 Registro de casos no resueltos:** el sistema debe almacenar en un *log* de incidencias todos los casos en los que la confianza no haya sido suficiente, para su posterior revisión por el administrador.
- **RF-2 Gestión del administrador:** la aplicación debe ofrecer una pestaña destinada al administrador que permita la gestión del conocimiento del sistema.
  - **RF-2.1 Añadir nuevos pares síntoma–especialidad:** el administrador debe poder añadir nuevas entradas al catálogo de datos, indicando los síntomas y la especialidad asociada. Estas entradas se almacenarán en un nuevo CSV adicional al original.
  - **RF-2.2 Visualización del log de incidencias:** el administrador debe poder consultar el listado de casos no resueltos almacenados en el *log*.

- **RF-2.3 Resolución de casos del log:** el administrador debe poder asignar una especialidad a los casos pendientes en el *log*, incorporando esos datos al CSV de aportaciones.
- **RF-2.4 Reentrenamiento de modelos:** el administrador debe poder regenerar los modelos de clasificación (TF-IDF, SVD, SVM y RNA) a partir del conjunto de datos actualizado mediante un único botón.

- **RF-3 Gestión de modelos de clasificación:** la aplicación debe integrar varios modelos de clasificación que puedan ser utilizados de forma independiente o conjunta.
  - **RF-3.1 Modelo TF-IDF:** el sistema debe disponer de un modelo basado en representación TF-IDF como línea base.
  - **RF-3.2 Modelo TF-IDF + SVD:** el sistema debe disponer de un modelo que aplique reducción de dimensionalidad mediante SVD sobre la representación TF-IDF.
  - **RF-3.3 Modelo SVM:** el sistema debe disponer de un clasificador SVM entrenado sobre la representación textual.
  - **RF-3.4 Modelo RNA:** el sistema debe disponer de un modelo basado en una red neuronal artificial.
  - **RF-3.5 Persistencia de modelos:** los modelos entrenados deben almacenarse localmente para no requerir reentrenamiento en cada arranque.

- **RF-4 Gestión de datos:** la aplicación debe gestionar el conjunto de datos de entrenamiento de forma persistente.
  - **RF-4.1 Carga del CSV original:** el sistema debe cargar el CSV original que contiene los pares síntomas–enfermedad–especialidad.
  - **RF-4.2 Mantenimiento de un CSV de ampliación:** las nuevas aportaciones realizadas por el administrador se almacenarán en un CSV adicional, manteniendo intacto el CSV original.
  - **RF-4.3 Combinación de fuentes para entrenamiento:** en el reentrenamiento, el sistema debe combinar el CSV original y el CSV de aportaciones.

Requisitos no funcionales

- **RNF-1 Ejecución *stand alone*:** la aplicación debe ejecutarse de forma totalmente local, sin depender de APIs externas ni servicios en la nube.

- **RNF-2 Privacidad por diseño:** ningún dato del paciente, sea texto o transcripción de voz, debe abandonar el equipo en el que se ejecuta la aplicación.

- **RNF-3 Rendimiento:** la aplicación debe ofrecer tiempos de respuesta razonables tanto en la fase de predicción como en el reentrenamiento de modelos.

- **RNF-4 Usabilidad:** la interfaz debe ser intuitiva y de baja curva de aprendizaje, permitiendo que un paciente sin formación técnica pueda interactuar sin dificultad.

- **RNF-5 Modularidad y mantenibilidad:** la arquitectura debe estar dividida en módulos (datos, preprocesado, modelos, predicción, interfaz) con interfaces claras que faciliten el mantenimiento y la extensión.

- **RNF-6 Reproducibilidad:** la aleatoriedad debe controlarse mediante semillas y la configuración experimental debe quedar registrada para que los resultados sean reproducibles.

- **RNF-7 Trazabilidad:** todas las predicciones por debajo del umbral, así como las acciones del administrador, deben quedar registradas para garantizar trazabilidad.

- **RNF-8 Portabilidad:** el sistema, desarrollado en Python con dependencias acotadas, debe poder ejecutarse en distintos sistemas operativos sin modificaciones sustanciales.

- **RNF-9 Escalabilidad funcional:** la arquitectura debe permitir incorporar nuevos modelos o nuevos canales de entrada (por ejemplo, telefonía) sin alterar el núcleo del sistema.

- **RNF-10 Eficiencia computacional:** el sistema debe priorizar configuraciones que ofrezcan un buen equilibrio entre precisión y coste computacional, en línea con el paradigma de IA frugal.

B.4. Especificación de requisitos

En esta sección se detallan los actores que interactúan con el sistema, el diagrama general de casos de uso y la descripción individual de cada uno de ellos.

Actores del sistema

A01	**Paciente**
Versión	1.0
Autor	Alumno
Descripción	Persona que utiliza la aplicación para describir sus síntomas y obtener una recomendación de especialidad médica.
Tipo	Usuario
Objetivo	Obtener una clasificación fiable de su sintomatología en una especialidad médica para orientar la solicitud de cita.
Responsabilidades	▪ Introducir sus síntomas mediante voz o texto. ▪ Aportar síntomas adicionales si el sistema lo solicita. ▪ Consultar la especialidad asignada.
Relaciones con casos de uso	CU-01, CU-02, CU-03, CU-04

Tabla B.1: A01 - Paciente.

A02	**Administrador**
Versión	1.0
Autor	Alumno
Descripción	Usuario con privilegios de gestión que mantiene el catálogo de datos y los modelos del sistema.
Tipo	Usuario
Objetivo	Garantizar la calidad y el mantenimiento del conocimiento que utiliza el sistema, incorporando nuevos datos y reentrenando los modelos cuando sea necesario.
Responsabilidades	▪ Añadir nuevos pares síntoma–especialidad. ▪ Revisar el *log* de incidencias. ▪ Asignar especialidad a casos no resueltos. ▪ Reentrenar los modelos.
Relaciones con casos de uso	CU-05, CU-06, CU-07, CU-08

Tabla B.2: A02 - Administrador.

A03	**Motor de clasificación**
Versión	1.0
Autor	Alumno
Descripción	Componente interno encargado de procesar el texto y devolver la predicción de especialidad mediante los modelos TF-IDF, SVD, SVM y RNA.
Tipo	Sistema
Objetivo	Clasificar texto libre en una especialidad médica con su correspondiente puntuación de confianza.
Responsabilidades	▪ Preprocesar el texto introducido. ▪ Aplicar los modelos de clasificación. ▪ Calcular el indicador de confianza.
Relaciones con casos de uso	CU-02, CU-03, CU-08

Tabla B.3: A03 - Motor de clasificación.

A04	**Módulo de reconocimiento de voz**
Versión	1.0
Autor	Alumno
Descripción	Componente local encargado de capturar el audio del paciente y transcribirlo a texto, sin envío a servicios externos.
Tipo	Sistema
Objetivo	Permitir la introducción de síntomas por canal de voz como alternativa al teclado.
Responsabilidades	▪ Capturar el audio del paciente. ▪ Transcribir la grabación a texto. ▪ Entregar la transcripción al motor de clasificación.
Relaciones con casos de uso	CU-01

Tabla B.4: A04 - Módulo de reconocimiento de voz.

A05	**Almacenamiento local (CSV y log)**
Versión	1.0
Autor	Alumno
Descripción	Sistema de persistencia local basado en ficheros CSV (datos originales y aportaciones) y en un *log* de incidencias para los casos no resueltos.
Tipo	Sistema
Objetivo	Garantizar la persistencia y trazabilidad de los datos del sistema sin recurrir a servicios externos.
Responsabilidades	▪ Almacenar el CSV original y el CSV de aportaciones. ▪ Registrar los casos no resueltos en el *log*. ▪ Persistir los modelos entrenados.
Relaciones con casos de uso	CU-03, CU-05, CU-06, CU-07, CU-08

Tabla B.5: A05 - Almacenamiento local.

Diagrama general de casos de uso

En la figura siguiente se representa el diagrama general de casos de uso del sistema, en el que se muestran las interacciones entre los actores principales (paciente y administrador) y los componentes internos.

Casos de uso

CU-01	**Introducir síntomas por voz**
Versión	1.0
Autor	Alumno
Actores	A01 Paciente, A04 Módulo de reconocimiento de voz
Requisitos asociados	RF-1, RF-1.2
Descripción	El paciente describe sus síntomas mediante voz y el sistema los transcribe a texto.
Precondición	La aplicación está abierta en la pestaña del paciente y el micrófono se encuentra disponible.
Acciones	1. El paciente accede a la pestaña Paciente. 2. Pulsa el botón de grabación. 3. Describe verbalmente sus síntomas. 4. El módulo de voz transcribe el audio a texto. 5. La transcripción se muestra en el campo de síntomas.
Postcondición	El campo de síntomas contiene la transcripción del audio, lista para su procesamiento.
Excepciones	Micrófono no disponible; transcripción vacía o ininteligible.
Importancia	Alta

Tabla B.6: CU-01 Introducir síntomas por voz.

CU-02	**Introducir síntomas por texto**
Versión	1.0
Autor	Alumno
Actores	A01 Paciente
Requisitos asociados	RF-1, RF-1.1
Descripción	El paciente introduce sus síntomas escribiéndolos directamente en el formulario.
Precondición	La aplicación está abierta en la pestaña del paciente.
Acciones	1. El paciente accede a la pestaña Paciente. 2. Escribe sus síntomas en el campo de texto. 3. Pulsa el botón de envío.
Postcondición	El sistema dispone del texto introducido para su procesamiento.
Excepciones	Campo vacío al pulsar el botón de envío.
Importancia	Alta

Tabla B.7: CU-02 Introducir síntomas por texto.

CU-03	**Obtener especialidad recomendada**
Versión	1.0
Autor	Alumno
Actores	A01 Paciente, A03 Motor de clasificación, A05 Almacenamiento local
Requisitos asociados	RF-1.3, RF-1.4, RF-3, RF-3.1, RF-3.2, RF-3.3, RF-3.4
Descripción	El sistema procesa los síntomas y devuelve la especialidad médica recomendada junto con su porcentaje de confianza.
Precondición	Existen modelos entrenados disponibles y el paciente ha introducido síntomas.
Acciones	1. El sistema preprocesa el texto introducido. 2. Aplica los modelos de clasificación (TF-IDF, SVD, SVM y RNA). 3. Calcula la especialidad predicha y su nivel de confianza. 4. Muestra al paciente la especialidad recomendada y, si procede, el porcentaje de acierto.
Postcondición	El paciente conoce la especialidad recomendada y el grado de confianza asociado.
Excepciones	Modelos no disponibles o corruptos; texto vacío tras el preprocesado.
Importancia	Alta

Tabla B.8: CU-03 Obtener especialidad recomendada.

CU-04	**Aportar síntomas adicionales y derivación**
Versión	1.0
Autor	Alumno
Actores	A01 Paciente, A03 Motor de clasificación, A05 Almacenamiento local
Requisitos asociados	RF-1.5, RF-1.6, RF-1.7
Descripción	Cuando la confianza de la predicción no alcanza el umbral, el sistema solicita síntomas adicionales y, si tampoco se obtiene una predicción fiable, deriva el caso a Atención General y lo registra en el *log*.
Precondición	Se ha realizado una primera predicción cuya confianza es inferior al umbral configurado.
Acciones	1. El sistema solicita al paciente síntomas adicionales. 2. El paciente añade información complementaria. 3. El sistema reevalúa la predicción. 4. Si la confianza es suficiente, devuelve la especialidad. 5. Si la confianza sigue siendo insuficiente, deriva a Atención General. 6. Registra el caso completo en el *log* de incidencias.
Postcondición	El paciente recibe una recomendación (especialidad concreta o Atención General) y el caso queda registrado en el *log* si fue dudoso.
Excepciones	El paciente cancela el proceso antes de completar la información adicional.
Importancia	Alta

Tabla B.9: CU-04 Aportar síntomas adicionales y derivación.

CU-05	**Añadir síntomas y especialidad**
Versión	1.0
Autor	Alumno
Actores	A02 Administrador, A05 Almacenamiento local
Requisitos asociados	RF-2, RF-2.1, RF-4.2
Descripción	El administrador añade nuevos pares síntoma–especialidad al catálogo de datos del sistema.
Precondición	El administrador ha accedido a la pestaña de administración.
Acciones	1. El administrador accede a la pestaña Administrador. 2. Selecciona la opción de añadir nuevo registro. 3. Introduce los síntomas y la especialidad asociada. 4. Confirma la operación. 5. El sistema almacena la nueva entrada en el CSV de aportaciones.
Postcondición	El CSV de aportaciones contiene una entrada nueva, lista para ser empleada en el próximo reentrenamiento.
Excepciones	Datos incompletos o especialidad fuera del catálogo.
Importancia	Alta

Tabla B.10: CU-05 Añadir síntomas y especialidad.

CU-06	**Visualizar log de incidencias**
Versión	1.0
Autor	Alumno
Actores	A02 Administrador, A05 Almacenamiento local
Requisitos asociados	RF-2.2
Descripción	El administrador consulta los casos no resueltos almacenados en el *log*.
Precondición	Existe al menos un caso no resuelto registrado.
Acciones	1. El administrador accede a la pestaña Administrador. 2. Selecciona la opción de revisión de incidencias. 3. El sistema muestra los registros del *log*.
Postcondición	El administrador dispone de la información necesaria para evaluar los casos pendientes.
Excepciones	*Log* vacío o no accesible.
Importancia	Media

Tabla B.11: CU-06 Visualizar log de incidencias.

CU-07	**Resolver casos del log**
Versión	1.0
Autor	Alumno
Actores	A02 Administrador, A05 Almacenamiento local
Requisitos asociados	RF-2.3, RF-4.2
Descripción	El administrador asigna manualmente una especialidad a los casos no resueltos y los incorpora al CSV de aportaciones.
Precondición	Existe al menos un caso no resuelto en el *log*.
Acciones	1. El administrador selecciona un caso del *log*. 2. Asigna la especialidad correspondiente. 3. Confirma la operación. 4. El sistema añade el caso al CSV de aportaciones y lo retira del *log*.
Postcondición	El caso queda incorporado al conjunto de datos y eliminado del *log*.
Excepciones	Especialidad no válida; fallo en la escritura del CSV.
Importancia	Alta

Tabla B.12: CU-07 Resolver casos del log.

CU-08	**Reentrenar modelos**
Versión	1.0
Autor	Alumno
Actores	A02 Administrador, A03 Motor de clasificación, A05 Almacenamiento local
Requisitos asociados	RF-2.4, RF-3.5, RF-4.3
Descripción	El administrador inicia el reentrenamiento de los modelos a partir del CSV original y del CSV de aportaciones.
Precondición	Existen los CSV de datos accesibles y los modelos pueden ser regenerados.
Acciones	1. El administrador pulsa el botón de reentrenamiento. 2. El sistema combina el CSV original y el CSV de aportaciones. 3. Reentrena los modelos TF-IDF, SVD, SVM y RNA. 4. Persiste los nuevos modelos en disco sustituyendo los anteriores. 5. Notifica al administrador la finalización del proceso.
Postcondición	Los modelos persistidos reflejan los datos actualizados, incluyendo las aportaciones del administrador.
Excepciones	Datos corruptos; error durante el entrenamiento; fallo de escritura.
Importancia	Alta

Tabla B.13: CU-08 Reentrenar modelos.

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

C.3. Diseño arquitectónico

C.4. Diseño procedimental

Apéndice D

Documentación técnica de programación

D.1. Introducción

D.2. Estructura de directorios

D.3. Manual del programador

D.4. Compilación, instalación y ejecución del proyecto

D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

E.1. Introducción

E.2. Requisitos de usuarios

E.3. Instalación

E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sosteniblidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Siglas

SVD *Singular Value Decomposition*. 1, 5, 6

SVM *Support Vector Machine*. 1, 5, 6

TF-IDF *Term Frequency – Inverse Document Frequency*. 1, 4, 5, 6

Glosario

standalone Sistema o aplicación que funciona de forma autónoma, sin depender de servicios externos, conexión a internet ni infraestructura adicional. 9

TruncatedSVD Variante de la Descomposición en Valores Singulares que opera directamente sobre matrices dispersas sin centrarlas, utilizada en NLP para reducción de dimensionalidad sobre matrices TF-IDF. 4, 5

Bibliografía

[1] Agencia Tributaria. Retenciones sobre rendimientos del trabajo. tablas de retención IRPF, 2026. [Internet; consultado mayo-2026].

[2] BOE. XVIII Convenio Colectivo Estatal de Empresas de Consultoría, Tecnologías de la Información y Estudios de Mercado y de la Opinión Pública, 2023. [Internet; consultado mayo-2026].

[3] scikit-learn developers. scikit-learn: Machine learning in Python, 2026. [Internet; consultado mayo-2026].

[4] Seguridad Social. Bases y tipos de cotización 2026, 2026. [Internet; consultado mayo-2026].

[5] Streamlit Inc. Streamlit documentation, 2026. [Internet; consultado mayo-2026].

[6] Zube. Zube — agile project management for GitHub, 2026. [Internet; consultado mayo-2026].